SISTEMAS DISTRIBUIDOS · 2027-1

Examen escrito en casa

Vence el sábado 21 de noviembre de 2026 a las 06:59. Individual, a mano, se entrega por push.

1. Qué es y para qué sirve
2. Qué entregas
3. Cómo se entrega
4. Cómo contestar
5. Banco de preguntas

TEMAS DEL BANCO

1. Por qué distribuir 9
2. Comunicación entre servicios 11
3. Contenedores, gateway y reparto 24
4. Copias y consistencia 12
5. Transacciones y fallas 14
6. Mensajería 13
7. Tiempo y orden 6
8. Bases de datos y caché 12
9. Blockchain y telemetría 5

1 Qué es y para qué sirve

Es tu acordeón para la entrevista del sábado 21 de noviembre. En la entrevista pasas tú solo y el profesor sortea 4 preguntas de este banco; puedes pedir que te cambie una. Lo que escribas aquí es lo que vas a estudiar para ese día.

2 Qué entregas

- Las 10 preguntas fijas, iguales para todos: `1.7`, `2.4`, `2.5`, `3.6`, `4.1`, `5.3`, `6.3`, `6.4`, `7.2` y `8.4`.
- 15 preguntas más del banco que tú elijas, de al menos 5 temas distintos, sin repetir las fijas ni los ejemplos.
- Todo a mano, con el número de cada pregunta.

3 Cómo se entrega

Un solo PDF escaneado en `entregas/apellido_nombre/examen/examen-escrito.pdf` de tu rama, por push, sin PR, antes del sábado 21 de noviembre de 2026 a las 06:59.

4 Cómo contestar

Contesta cada pregunta en 4 a 6 líneas, con tus palabras: qué es, qué ganas, qué pierdes y cuándo lo usarías. Así se ven dos respuestas buenas; estas dos no se entregan.

Ejemplo 1

PREGUNTA. ¿Qué diferencia hay entre escalar vertical y horizontalmente, y cuándo usarías cada uno?

RESPUESTA

Escalar vertical es darle más CPU, memoria o disco a la misma máquina; escalar horizontal es agregar máquinas y repartir la carga entre ellas. Vertical es simple, porque no pide código nuevo ni coordinación, pero tiene techo y una sola máquina es punto único de falla. Horizontal casi no tiene techo y una copia caída no tumba el servicio, pero pide servidores sin estado y un balanceador, y trae fallas parciales. Escalaría horizontal la capa web sin estado, porque clonarla detrás de un balanceador es barato, aunque pierdo simplicidad de operación. La base la crecería vertical mientras alcance, porque partirla cuesta mucho más.

Ejemplo 2

PREGUNTA. ¿Cuándo elegirías una base SQL y cuándo una NoSQL?

RESPUESTA

Una base relacional (SQL) guarda tablas con esquema, joins y transacciones de todo o nada sobre varias filas; una NoSQL guarda, por ejemplo, documentos o pares llave y valor, pensada para un acceso simple y conocido. Elegiría NoSQL cuando leo por llave o el documento completo, los datos cambian de forma o las escrituras ya no caben en una máquina: el catálogo de una tienda, que se lee entero y trae atributos distintos por producto. Elegiría SQL cuando hay reglas entre filas y operaciones de todo o nada sobre varias cosas, como pedido y pago, o consultas que todavía no conozco. Con NoSQL gano escala y flexibilidad, pero el esquema no desaparece: lo revisa mi código. Por eso uso relacional por defecto y NoSQL solo cuando puedo nombrar el acceso o la carga que la relacional no aguanta.

5 Banco de preguntas

Nueve temas. Escribe el número de la pregunta junto a cada respuesta.

TEMA 1 Por qué distribuir 9

1.1	¿Cuándo distribuirías un sistema en varias máquinas, y cuándo lo dejarías en una sola?
1.2	¿Un sistema es más confiable o menos por estar distribuido en varias máquinas?
1.3	Explica qué es la elasticidad y en qué se distingue de la escalabilidad.
1.4	¿Qué necesita un servicio para poder agregarle y quitarle copias en minutos?
1.5	Explica qué es una falla parcial y por qué un programa que corre en una sola máquina normalmente no la tiene.
1.6	¿Qué diferencia hay entre un monolito modular y una arquitectura basada en servicios, y cuándo elegirías cada uno?
1.7	¿Qué diferencia hay entre un monolito y microservicios, y cuándo usarías cada uno?
1.8	Si un monolito sin estado puede correr en diez copias, ¿qué ganan de verdad los microservicios al escalar?
1.9	¿Qué se rompe primero si varios servicios comparten la misma base de datos?

TEMA 2 Comunicación entre servicios 11

2.1	¿Qué diferencia hay entre REST y gRPC, y cuándo elegirías gRPC?
2.2	¿Por qué no expondrías gRPC a clientes que no controlas?
2.3	¿Qué tiene de mal esta afirmación de una guía de entrevista: "gRPC es entre cinco y diez veces más rápido que REST"?
2.4	¿Qué significa que una operación sea idempotente, y cómo harías que reintentar un cobro no cobre dos veces?
2.5	¿Qué pasa si la llamada de tu servicio a una pasarela de cobro vence su timeout? ¿Qué sabes y qué no sabes en ese momento?
2.6	Llegan al mismo tiempo dos peticiones de cobro con la misma llave de idempotencia: ¿qué hace el servidor?
2.7	En una cadena de tres servicios, ¿cómo evitas que el último siga trabajando cuando el primero ya se rindió?
2.8	Explica el modelo de madurez de Richardson y qué gana una API cuando usa los verbos y códigos HTTP según su significado.
2.9	En el modelo de madurez de Richardson, ¿cuándo valdría la pena llegar al nivel 3?
2.10	¿Qué diferencia hay entre que dos servicios se comuniquen de forma síncrona o asíncrona, y cuándo elegirías cada una?
2.11	¿Cómo se entera el usuario de que su pedido, procesado de forma asíncrona, ya quedó?

TEMA 3 Contenedores, gateway y reparto

3.1	¿Qué pasa con los archivos que un contenedor escribió dentro de sí cuando lo borras y levantas otro con la misma imagen?
3.2	¿Por qué conviene que la imagen de un contenedor sea inmutable, si eso te obliga a reconstruirla por cada cambio?
3.3	¿Cuándo te basta un orquestador de un solo host, como Docker Compose, y cuándo necesitas uno de clúster, como Kubernetes?
3.4	En Kubernetes, si muere uno de los tres Pods de un Deployment, ¿quién lo repone y cómo lo siguen encontrando sus clientes?
3.5	Explica qué cuenta como configuración en una aplicación de doce factores y dónde debe vivir.
3.6	¿Qué diferencia hay entre un API gateway y un balanceador de carga, y cuándo pondrías cada uno?
3.7	¿Pondrías en el API gateway la lógica de negocio o las llamadas entre tus servicios internos?
3.8	¿Qué pasa si una instancia de un servicio muere de golpe, sin avisar al registro de servicios?
3.9	Cuando muere una instancia, ¿por qué no basta con que el registro de servicios reaccione rápido, si cada cliente guarda su propia copia de la lista?
3.10	¿Qué diferencia hay entre repartir carga por turnos (round robin) y por menos conexiones, y cuándo elegirías cada uno?
3.11	Los servidores detrás de tu balanceador son de distinto tamaño: ¿qué cambias en el reparto?
3.12	¿Cuándo aceptarías sticky sessions, que atan a cada usuario a un mismo servidor, y cuándo no?
3.13	El balanceador ata a cada usuario a un servidor según su IP, y muchos llegan desde la misma red de oficina: ¿qué le pasa al reparto?
3.14	¿Qué diferencia hay entre replicar y particionar los datos, y cuándo usarías cada uno?
3.15	Si el cuello de botella de tu base son las escrituras, ¿te ayudan las réplicas?
3.16	¿Qué tiene de mal esta afirmación: "partir una base en shards le agrega redundancia"?
3.17	Si partes una base en shards, ¿qué sí gana el sistema en cuanto a fallas?
3.18	¿Qué diferencia hay entre particionar por rangos de la clave y por hash de la clave, y cuándo elegirías cada uno?
3.19	Particionas una tabla por fecha de creación: ¿qué shard recibe las escrituras de hoy?
3.20	¿Qué pasa si una sola clave, como la cuenta de una celebridad, recibe muchísimo más tráfico que las demás en una base particionada?
3.21	En una base particionada, ¿cómo te enteras de que tienes una clave caliente antes de que se caiga su shard?
3.22	Explica cómo llega una petición al shard correcto cuando los datos están repartidos en varios shards.
3.23	En una base partida en shards, ¿qué pasa si quien guarda el mapa de qué clave vive en qué shard se cae o tiene una versión vieja?
3.24	¿Qué diferencia hay entre un contenedor y una máquina virtual, y cuándo usarías cada uno?

TEMA 4 Copias y consistencia

12

4.1	Explica el teorema CAP y qué implica para el diseño de un sistema con datos replicados.
4.2	En un sistema con datos replicados, cuando la red está sana, ¿te cuesta algo pedir consistencia fuerte?
4.3	¿Qué diferencia hay entre consistencia fuerte (linealizable) y consistencia eventual, y cuándo elegirías cada una?
4.4	Con consistencia eventual, ¿cuánto tarda una réplica en ponerse al día, y quién te lo garantiza?
4.5	¿Qué diferencia hay entre replicación síncrona y asíncrona, y cuándo elegirías cada una?
4.6	¿Qué tiene de mal esta afirmación: "si el servidor primario falla, una de las réplicas toma su lugar automáticamente como nuevo primario, sin perder nada"?
4.7	Para detectar que el servidor primario cayó, ¿por qué un timeout muy corto puede ser peor que uno largo?
4.8	¿Qué pasa si alguien borra por error una tabla en producción y tu base solo tiene réplicas, sin respaldos?
4.9	Si tienes que restaurar tu base desde un respaldo, ¿cuántos datos aceptas perder y cuánto tiempo aceptas estar caído?
4.10	¿Cómo elige Raft a un líder, y por qué no puede haber dos líderes en el mismo término?
4.11	En Raft, ¿para qué sirve que cada nodo espere un tiempo distinto antes de postularse?
4.12	¿Raft tolera más fallas que Paxos o es más rápido? ¿Qué gana entonces?

TEMA 5 Transacciones y fallas

14

5.1	Explica qué garantiza una transacción ACID y por qué esa garantía se pierde cuando una operación cruza varios servicios, cada uno con su base.
5.2	¿Qué nivel de aislamiento usa por omisión una base relacional como PostgreSQL, y qué anomalía deja pasar?
5.3	¿Qué diferencia hay entre una saga y el commit en dos fases, y cuándo elegirías cada uno para una operación que cruza varias bases?
5.4	En una saga, el pago pasó y el inventario falló, y la compensación del pago también falla: ¿qué hace tu sistema?
5.5	En un commit en dos fases, ¿qué pasa si el coordinador se cae después de que los participantes votaron que sí y antes de comunicarles la decisión?
5.6	¿Por qué el commit en dos fases no es un algoritmo de consenso, si los dos buscan que todos decidan lo mismo?
5.7	¿Qué tiene de mal esta afirmación: "consistencia significa que los datos del sistema son iguales en todos los componentes, en todos los casos"?
5.8	Mientras una saga va a medias, ¿qué consistencia pierde el sistema y cuál conserva?
5.9	Explica el patrón circuit breaker: qué problema evita y cómo se comporta en cada uno de sus estados.
5.10	En un circuit breaker, ¿qué errores deben abrir el circuito y cuáles no?

5.11	¿Cuándo reintentarías una llamada que falló hacia otro servicio, y cuándo no?
5.12	Si cada capa de tu sistema reintenta por su cuenta, ¿qué le pasa a la base cuando ya está saturada?
5.13	¿Qué pasa con tu servicio si un proveedor externo del que dependes no se cae, sino que empieza a tardar treinta segundos en cada respuesta?
5.14	Si se cae el proveedor de pagos, ¿qué funciones de una tienda en línea pueden seguir y cuáles deben rechazarse?

TEMA 6 Mensajería

13

6.1	¿Cuándo pondrías una cola de mensajes entre dos servicios, y cuándo no?
6.2	¿Qué haces si una cola de mensajes crece sin parar?
6.3	Cola punto a punto, pub/sub o log: ¿qué diferencia hay entre los tres y cuándo elegirías cada uno?
6.4	Explica por qué un broker no puede prometerte que cada mensaje se entregue exactamente una vez, y qué puedes prometer en su lugar.
6.5	Un servicio nuevo necesita los eventos de la semana pasada: ¿qué usas?
6.6	Quieres que el efecto de cada mensaje ocurra una sola vez, pero el efecto es un cobro en un sistema externo que no controlas: ¿qué haces?
6.7	¿Qué diferencia hay entre un evento y un comando, y cuándo usarías cada uno?
6.8	Si nadie consume un evento, ¿es un error?
6.9	¿Qué diferencia hay entre publicar un evento justo después del commit y usar el patrón outbox, y cuándo elegirías cada uno?
6.10	Si usas el patrón outbox, ¿por qué el consumidor tiene que seguir siendo idempotente?
6.11	¿Qué tiene de mal esta afirmación de una guía de entrevista: "lo asíncrono es excelente para el rendimiento y la tolerancia a fallas, solo que es más complicado"?
6.12	Si publicar en el broker también es una llamada remota, ¿qué cambia y qué no cuando pasas de una llamada directa a una cola?
6.13	¿Qué pasa si un consumidor se cae a mitad de procesar un mensaje, y para qué sirve una cola de mensajes muertos (DLQ)?

TEMA 7 Tiempo y orden

6

7.1	¿Qué pasa si un sistema decide el orden de dos eventos ocurridos en máquinas distintas comparando la hora que marcó el reloj de cada una?
7.2	Explica la relación "sucedió antes" y cómo la respeta un reloj de Lamport.
7.3	¿Qué reloj usas para medir cuánto tardó una llamada?
7.4	Con relojes de Lamport, si dos eventos de procesos distintos tienen el mismo número, ¿cómo sacas un orden que todos compartan?
7.5	¿Qué diferencia hay entre un reloj de Lamport y uno vectorial, y cuándo pagarías el costo del vectorial?
7.6	En un chat de tres personas, ¿por qué la entrega FIFO no basta para que nadie vea la respuesta antes que la pregunta?

TEMA 8 Bases de datos y caché 12

8.1	¿Qué diferencia hay entre normalizar y desnormalizar, y cuándo duplicarías datos a propósito?
8.2	Si duplicas un dato a propósito, ¿quién actualiza la copia cuando cambia el original, y qué pasa si falla a la mitad?
8.3	Explica qué es la persistencia políglota y qué te cuesta usar varias bases de datos a la vez.
8.4	¿Cuándo pondrías una caché frente a tu base de datos, y qué no cachearías nunca?
8.5	¿Qué le pasa a tu base de datos si la caché se cae en la hora pico?
8.6	Explica qué es invalidar una caché y por qué es difícil hacerlo bien.
8.7	¿Por qué borrar la llave de la caché antes del commit también puede dejar un valor viejo?
8.8	¿Qué pasa cuando expira una entrada de caché muy popular? ¿Y si una sola llave concentra casi todo el tráfico?
8.9	Mientras se reconstruye una entrada de caché muy popular, ¿qué ven los usuarios que esperan?
8.10	¿Qué tiene de mal esta afirmación de una guía de entrevista: "con write-through, los datos de la caché nunca están viejos"?
8.11	Con una caché write-through, ¿qué pasa si alguien corrige un precio directo en la base?
8.12	¿Cuántos motores de bases de datos distintos puede operar bien un equipo de tres personas, y cómo lo decides?

TEMA 9 Blockchain y telemetría 5

9.1	¿Cuándo necesitas una blockchain y cuándo te basta una base de datos?
9.2	¿En qué se diferencia el consenso de Bitcoin del de Raft, y por qué Raft no sirve en una red abierta?
9.3	¿Qué diferencia hay entre un consenso que tolera nodos que se caen, como Raft, y uno que tolera nodos que mienten, y cuándo usarías cada uno?
9.4	Explica qué pregunta responde cada señal de observabilidad: logs, métricas y trazas.
9.5	¿Cómo sigues con trazas una petición cuando cruza una cola entre dos servicios?

Preguntas sobre la guía: en el canal del curso.